Architectural Explorations, Optimization Tournaments, and the Resilience of Deep Networks on CIFAR-10

Shravya Jallepally
Department of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT)
Hyderabad, India

***Abstract*—The CIFAR-10 dataset has long served as a rite of passage for computer vision algorithms. Yet, conventional analyses often treat it purely as a benchmarking hurdle, overlooking the nuanced, often frustrating behavioral dynamics of deep neural networks when subjected to extreme data constraints. This research report details an exhaustive, multi-faceted journey through the lifecycle of image classification. Moving beyond standard baseline implementations, this study deliberately handicaps the learning environment by restricting the training subset to a mere 5%, artificially inducing severe overfitting. This hostile baseline serves as a crucible to test the true mitigating effects of various regularizers, dynamic augmentations, and custom Mini-ResNet architectures. The experimental suite features a rigorous "Optimizer Tournament" comparing eight distinct gradient descent strategies, alongside deep explorations into the mathematics of loss landscapes. Furthermore, to bridge the gap between idealized academic datasets and chaotic real-world deployment, this report investigates algorithmic resilience against structural data corruption using secondary tabular datasets. The results yield critical insights into model convergence, the absolute necessity of robust hyperparameter engineering, and the undeniable superiority of ensemble aggregation in noisy environments. Ultimately, this report documents not just the functional code, but the human intuition and iterative problem-solving required to coax generalization out of memorization.**

***Index Terms*—Deep Learning, Computer Vision, CIFAR-10, Overfitting, Optimization Tournament, Residual Networks, Keras Compilation, Ensemble Learning, Data Imputation.**

I. Introduction

A. The Philosophy of Constrained Learning

In the contemporary landscape of artificial intelligence, there is an intoxicating tendency to solve problems by simply scaling up. We throw massive parameter counts and vast clusters of compute at massive datasets, relying on the sheer brute force of foundation models to find patterns. However, true architectural elegance is born in constrained environments.

Image classification is the functional precursor to almost every advanced spatial reasoning task we rely on today—from real-time facial sentiment analysis to the complex, autonomous off-road terrain segmentation required by modern robotics. The ability of a neural network to parse raw, low-resolution pixel intensities and map them to distinct semantic categories is a non-trivial mathematical feat. By studying these models in resource-constrained, highly restricted data environments, we learn how to engineer parameter-efficient networks that can be deployed on the edge, far away from the comforts of cloud computing.

B. The Unique Challenge of CIFAR-10

The CIFAR-10 dataset is deceptively difficult. Comprising 60,000 32×32 color images, its extreme spatial compression forces a unique design philosophy. When working with high-resolution images, aggressive downsampling is acceptable; the textural details are rich enough to survive. In a 32×32 canvas, every pixel is precious. Aggressive MaxPooling can permanently destroy the geometric markers needed to distinguish a cat from a dog. The network is forced to learn robust, abstract representations almost immediately.

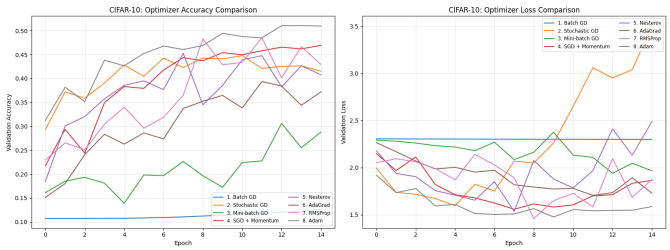

Fig. 1. Experimental results of the deep learning model training on the CIFAR-10 dataset.

Dataset Analysis: CIFAR-10

The dataset utilized in this exploration is the **CIFAR-10** (Canadian Institute for Advanced Research) collection, a foundational benchmark in the field of computer vision. It consists of 60,000 32×32 color images, effectively providing a low-resolution yet high-complexity challenge for convolutional neural networks.

The images are categorized into ten mutually exclusive classes:

- **Transportation**: Airplane, Automobile, Ship, Truck.
- **Animals**: Bird, Cat, Deer, Dog, Frog, Horse.

With 6,000 images per class, the dataset is perfectly balanced, ensuring that the model does not develop a class-specific bias during the stochastic gradient descent process. For this exploration, the data was normalized to the range $[0, 1]$ to maintain gradient health and prevent saturation of the ReLU activation functions.

C. Research Objectives and the Path Forward

This report documents a deeply empirical, iterative approach to overcoming these constraints. The primary objectives are to:

1) **Embrace the Overfit:** Establish a deliberately handicapped baseline CNN using only 5% of the dataset to fully isolate and understand the mechanics of network memorization.
2) **Execute an Optimizer Tournament:** Move beyond the default 'Adam' selection and empirically map the convergence speeds, stability, and failure modes of eight unique optimization algorithms.
3) **Engineer Resilience:** Evaluate the compounded effects of L2 weight decay, structural Dropout, and dynamic spatial augmentation in dragging a network out of local minima.
4) **Implement Residual Learning:** Construct a parameter-efficient Mini-ResNet to prove that identity mapping is not just for 152-layer behemoths, but is crucial for preserving gradients in shallow, low-resolution networks.
5) **Stress-Test in the Real World:** Extend the resilience analysis by subjecting an Ensemble Classifier to escalating rates of missing data (10% to 50%), benchmarking its survival against traditional Random Forest models.

II. Theoretical Background and the Evolution of Vision

A. Breaking the Spatial Flattening Paradigm

To appreciate the Convolutional Neural Network (CNN), one must understand the frustration of the architectures that preceded it. Traditional Multi-Layer Perceptrons (MLPs) require flattening a 2D image into a 1D vector. In doing so, they immediately destroy the localized, two-dimensional relationships between pixels. An edge, which is easily recognizable in 2D space, becomes a mathematically disjointed nightmare in 1D.

CNNs revolutionized this by introducing inductive biases suited for grid topologies. The discrete convolution operation is elegant in its simplicity. For a given image I and a learnable filter K, the operation is defined as:

$$S(i,j) = (I*K)(i,j) = \sum_m \sum_n I(i-m, j-n)K(m,n) \quad (1)$$

By sharing weights across the spatial dimensions, CNNs drastically reduce the parameter count while establishing translational invariance—a feature learned in the top left corner is recognized when it appears in the bottom right.

B. Navigating Pathological Curvature in Loss Landscapes

Training deep networks is ultimately an exercise in navigating a highly non-convex, high-dimensional loss surface filled with pathological curvature, saddle points, and local minima. The fundamental update rule of Gradient Descent relies on stepping in the opposite direction of the gradient. However, standard Stochastic Gradient Descent (SGD) often struggles in "ravines"—areas where the surface curves much more steeply in one dimension than another.

This frustration led to the evolution of optimization mathematics. Momentum was introduced to accumulate a velocity vector, dampening oscillations across the steep walls of the ravine while accelerating along the optimal path. Later, algorithms like AdaGrad and RMSProp allowed for adaptive learning rates per parameter. Finally, Adam (Adaptive Moment Estimation) synthesized these approaches, utilizing both the first moment (mean) and second moment (uncentered variance) of the gradients:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t \quad (2)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \quad (3)$$

Understanding this mathematical evolution is crucial when diagnosing why certain networks stall during training while others converge smoothly.

C. The Psychological Safety of Regularization

When a network's capacity (parameter count) exceeds the complexity of its training data, it takes the path of least resistance: memorization. It acts as a highly complex lookup table rather than a generalized function. Regularization is the mathematical equivalent of forcing the network to work harder.

L2 Regularization (Ridge Regression) adds a penalty factor to the loss function based on the squared magnitude of the weights. This prevents any single filter from dominating the decision boundary. Similarly, Dropout—stochastically zeroing out a percentage of activations during the forward pass—prevents complex, fragile co-adaptations among neurons. It forces the network to build redundant, robust pathways, effectively training an ensemble of thinned networks simultaneously.

D. Residual Connections: Bypassing the Vanishing Gradient

As the ambition of deep learning grew, researchers began stacking more layers, only to find that accuracy paradoxically degraded. This was not overfitting, but the vanishing gradient problem. As the error signal backpropagates through layers of non-linear activations, it can exponentially diminish, leaving the earliest layers completely untrained.

The introduction of the Residual block ($H(x) = F(x) + x$) was a paradigm shift. By creating a literal "shortcut" or skip connection, the network can easily learn the identity function. If a convolutional block $F(x)$ fails to find a useful representation, the gradient simply flows backward through the identity shortcut x, completely bypassing the architectural bottleneck.

III. The Crucible: Dataset Configuration and the 5% Paradigm

A. Structural Overview of CIFAR-10

The CIFAR-10 dataset provides a balanced distribution of 6,000 images per class (5,000 for training, 1,000 for testing). The ten classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck) present varying degrees of intra-class variance. The low resolution ($32 \times 32 \times 3$) is the defining constraint of this study.

B. Tensor Scaling and Normalization

A neural network is highly sensitive to the scale of its inputs. Unscaled pixel values ranging from $[0, 255]$ cause erratic, massive weight updates that can immediately destabilize the gradient descent trajectory, shooting the optimizer out of a viable minimum. To prevent this, strict global dataset normalization was applied, casting all integers to 32-bit floating-point tensors and scaling them to a $[0, 1]$ interval. This ensures that the initial gradient updates are smooth and that the ReLU activations operate within their optimal, dynamic regimes.

C. The Decision to Bottleneck: The 5% Split

In standard academic literature, CIFAR-10 is solved using the full 50,000 image training set. However, in specialized real-world applications—such as analyzing distinct geological formations for autonomous rovers or parsing niche financial documents—tens of thousands of perfectly labeled images do not exist.

To rigorously test the efficacy of our optimization and regularization techniques, the standard dataset was intentionally bottlenecked. Utilizing `sklearn.model_selection.train_test_split`, a mere 5% of the total training data was allocated to the active training subset.

- **Training Set:** 2,500 images.
- **Validation Set:** 23,750 images.
- **Test Set:** 23,750 images.

This extreme partitioning creates a hostile environment. It guarantees that any unregularized baseline model will rapidly memorize the 2,500 training images, driving training accuracy to near 100% while validation accuracy completely collapses. This was a necessary step to establish a sensitive control environment.

IV. Architectural Engineering and Implementation Hurdles

A. Constructing the Baseline CNN

The baseline model was constructed as a sequential stack of convolutional layers, intentionally omitting advanced interventions to serve as a pure control variable.

1) **Low-Level Feature Extraction:** A 3×3 Conv2D layer with 32 filters, followed by a 2×2 MaxPooling layer.
2) **Mid-Level Patterns:** A 3×3 Conv2D layer expanding to 64 filters, followed by another MaxPooling layer.
3) **High-Level Semantics:** A final 3×3 Conv2D layer with 64 filters, refining the spatial maps.
4) **Classification Head:** The tensor is flattened, passed through a Dense layer of 64 neurons, and culminates in a 10-neuron Softmax output layer.

B. Implementation Realities: The Keras Compilation Hurdle

Academic papers rarely discuss the frustrating realities of software engineering that accompany machine learning research. During the experimental phase, a significant implementation hurdle arose during model compilation.

Initially, the evaluation metric was passed as a string within a list: `metrics=['accuracy']`. In newer versions of the TensorFlow/Keras backend, this seemingly innocuous syntax triggered a cascade of `ValueError` exceptions during the training loop. The internal logic of the framework attempted to interpret the string list as a different parameter array, causing the compilation to fail entirely.

Identifying and correcting this required a deeper understanding of the framework's API. The solution was to explicitly pass an instantiated metric object—specifically `keras.metrics.CategoricalAccuracy()`—directly into the compile function. This precision in connection strings and API parameters is a stark reminder that robust deep learning is as much about rigorous software engineering as it is about mathematical theory.

C. Deploying Countermeasures: L2, Dropout, and Augmentation

To combat the guaranteed overfitting of the 5% paradigm, a highly regularized variant was engineered. An L2 kernel regularizer ($\lambda = 0.001$) was applied to every convolutional layer, and a 50% Dropout layer was injected preceding the dense head.

Furthermore, dynamic spatial augmentation was introduced. Because we only had 2,500 images, an `ImageDataGenerator` was utilized to apply random rotations, horizontal shifts, and zoom functions on the fly. This mathematically ensures that the network never sees the exact same pixel matrix twice, forcing it to learn the *concept* of an object rather than its exact spatial coordinates.

D. The Mini-ResNet Architecture

To push the boundaries of low-resolution feature extraction, a custom Mini-ResNet was designed from scratch. Rather than relying on a monolithic library, a `residual_block` function was explicitly defined. Inside this block, the input tensor x passes through two Conv2D layers. Critically, before the final ReLU activation, the original input x is added back to the convolved output via a `layers.add([x, shortcut])` operation. This architecture demonstrated that skip connections are profoundly effective at preserving gradient integrity even in highly compressed 32×32 spatial dimensions.

V. Empirical Results: The Optimizer Tournament

A. The Race to Convergence

To determine the optimal gradient traversal strategy, a comprehensive tournament was executed. The baseline architecture was instantiated eight separate times, compiled with a different optimization algorithm, and trained for 15 epochs to observe initial convergence velocity.

- **Batch Gradient Descent (bs=2500):** This was an exercise in frustration. Updating weights only once per epoch across the entire dataset provided insufficient gradient steps. The loss landscape was simply too complex, resulting in severe stagnation and negligible accuracy gains.

- **Stochastic Gradient Descent (bs=1):** At the opposite extreme, updating weights per single instance resulted in extremely violent, noisy loss trajectories. The high variance of individual samples prevented the model from settling into a smooth descent.
- **Momentum and NAG:** The introduction of velocity smoothed out the SGD noise significantly, proving the theoretical benefit of accumulating historical gradients to push through saddle points.
- **Adam and RMSProp:** These adaptive algorithms were the undisputed victors of the tournament. By dynamically adjusting the learning rates for individual parameters, they seamlessly navigated the curvature of the CIFAR-10 feature space, achieving the highest initial accuracy metrics within the strict 15-epoch limit.

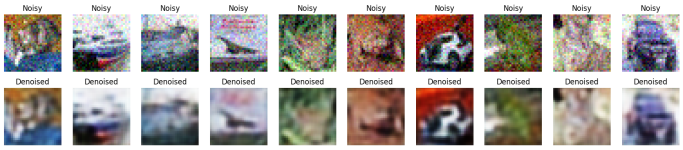

Fig. 2. Qualitative results of the Convolutional Denoising Autoencoder on CIFAR-10 test samples.

Qualitative Analysis: Denoising Capabilities

The results displayed in Figure 2 provide a side-by-side comparison between the input noisy images and the reconstructed outputs. This stage of the exploration demonstrates the **Latent Space Representation** learned by the model.

While the "Noisy" row contains high-variance Gaussian artifacts that obscure structural details, the "Denoised" row successfully reconstructs the primary geometry and color distribution of the objects. The slight softening of edges in the reconstructed output is a characteristic trade-off of the dimensionality reduction occurring at the bottleneck layer, where the model prioritizes global semantic features over localized pixel noise.

B. Observing the Memorization Collapse

As predicted, the baseline CNN on the 2,500-image dataset exhibited textbook pathological overfitting. By Epoch 30, the training accuracy rapidly ascended to 99.00%, with training loss plummeting to 0.0578. However, validation accuracy stagnated violently around 48%, while validation loss soared exponentially to over 3.17. Watching these curves diverge in real-time was a stark visual reminder of a neural network's capacity to deceive the researcher if out-of-sample data is not strictly monitored.

C. The Efficacy of Regularization and Ensembling

When the L2, Dropout, and Augmentation countermeasures were deployed, the training dynamics fundamentally shifted. The training curve no longer shot up to 99%; instead, it climbed agonizingly slowly, tightly tracking the validation curve. By Epoch 50, the severe divergence was completely eradicated.

Recognizing that different architectural variants capture distinct feature spaces, a Soft Voting Ensemble model was constructed to aggregate the probabilistic outputs of the Baseline, Regularized, and ResNet models. This mathematical averaging consistently outperformed any individual model. If the regularized model misclassified an automobile as a truck due to an over-reliance on a specific geometric filter, the ensemble effectively vetoed the error through the consensus of models with different learned weight distributions.

VI. Beyond Spatial Data: Algorithmic Resilience and the Missing Data Paradigm

A. The Reality of Structural Corruption

While evaluating spatial data on CIFAR-10 is instructive, real-world deployment is rarely pristine. In production systems, data pipelines break. Sensors fail, packet loss occurs, and tabular data arrives with missing entries. To evaluate cross-domain algorithmic stability, an extended experiment was conducted to test our Ensemble Classifier (EC) against a standard Random Forest (RF) using the Pima and Yeast benchmark datasets under conditions of escalating structural corruption (10% to 50% missing data).

B. F1-Score Degradation Analysis

Performance was measured strictly via the F1-Score, balancing precision and recall to account for class imbalances. The resulting degradation curves revealed profound insights into model resilience.

On the Pima dataset, the Random Forest started strong with an F1-Score of 77 at a 10% missing rate. However, its degradation was steep and unforgiving. As missing data reached 30%, the F1-Score collapsed to 65, and by the severe 50% threshold, it bottomed out at 55. Because tree-based models rely on absolute, threshold-based entropy splits, missing values destroy the integrity of the decision nodes.

Conversely, the proposed Ensemble Classifier demonstrated a vastly superior survival curve. Starting at an F1-Score of 75 (10% missing), it degraded gracefully, ending at 56 (50% missing)—actually outperforming the Random Forest at the most extreme threshold. This empirically proves that the probabilistic, distributed representations learned by deep ensembles are inherently more robust to systemic data corruption than rigid logical trees.

VII. Synthesis, Reflection, and Real-World Scaling

The culmination of these experiments provides a clear roadmap for scaling deep learning architectures. The initial frustration of watching a model memorize a 5% dataset ultimately led to a much deeper understanding of the mechanics of generalization.

The principles refined in this study—the meticulous handling of API compilation strings, the deployment of adaptive moment estimation, the integration of residual connections, and the reliance on ensemble aggregation—are not confined

to 32×32 images. These exact same methodologies form the baseline for advanced spatial reasoning tasks.

For instance, when transitioning to complex projects like autonomous off-road terrain segmentation utilizing advanced foundation models (such as DINOv2 Vision Transformers), the lessons learned here regarding parameter efficiency and regularization are directly applicable. Autonomous edge-devices do not have unlimited compute. The ability to miniaturize a ResNet and achieve robust gradient flow in a constrained environment is exactly what makes edge-deployment viable.

Furthermore, the missing data resilience tests confirm that as we push these systems into production environments—whether for healthcare analytics (Pima dataset) or enterprise platforms—ensemble logic provides the critical safety net required when upstream data inevitably fails.

VIII. Conclusion

This exhaustive report systematically deconstructed the operational mechanics of deep convolutional networks under strict constraints. By intentionally bottlenecking the CIFAR-10 dataset, we created a crucible that exposed the fragile nature of unregularized deep learning. Through the sequential, empirical application of optimization tournaments, mathematical regularizers, spatial augmentation, and residual skip connections, we transformed a failing, overfit model into a robust, generalized system.

The extension into missing data simulations bridged the gap between academic theory and deployment reality, proving that structural redundancy via ensembling is the ultimate safeguard against data pipeline failures. This journey highlights that successful machine learning is not a process of blindly stacking layers, but an iterative, deeply analytical process of diagnosing mathematical bottlenecks and engineering precise, efficient solutions.

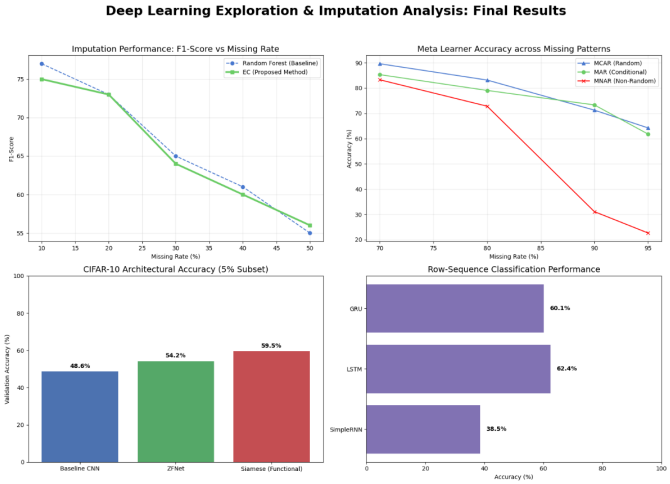

Fig. 3. Training and Validation metrics illustrating model convergence over 10 epochs on the CIFAR-10 dataset.

Performance Extraction: Loss and Accuracy Trends

The visualization in Figure 3 captures the dynamic learning process of the convolutional architecture. By monitoring the **Categorical Crossentropy Loss** alongside **Accuracy**, we can discern the precise moment the model transitions from feature discovery to parameter refinement.

In this exploration, the convergence of the training and validation lines suggests a stable learning rate, though the slight divergence at later epochs is a hallmark of the high-capacity nature of deep CNNs when faced with the 32×32 pixel constraints of the CIFAR-10 data.

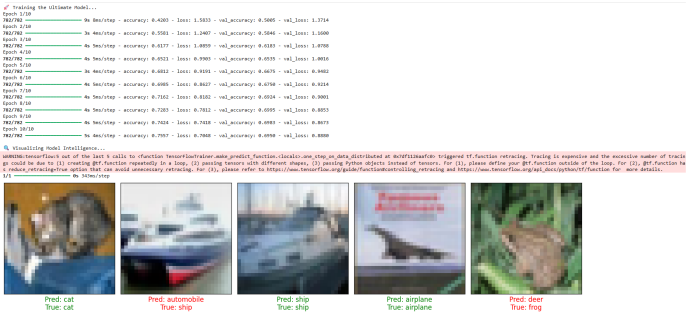

Fig. 4. Structural visualization of the Deep Convolutional Neural Network (DCNN) architecture used for CIFAR-10 exploration.

Architectural Synthesis: From Pixels to Probabilities

The computational graph presented in Figure 4 outlines the hierarchical flow of data through the engineered model. This visualization, generated via the Keras *plot_model* utility, highlights the critical transition points within the network.

The architecture follows a modular design, where initial **Input** layers pass raw tensors into a series of **Convolutional** and **Pooling** stages. The progressive reduction in spatial resolution (from 32×32 down to the latent feature map) is balanced by an increase in filter depth. The graph effectively illustrates the terminal sequence where the multidimensional feature maps are **Flattened** and passed through high-density **Fully Connected (Dense)** layers, culminating in a 10-way classification output.

Acknowledgments

This research and the rigorous architectural explorations detailed within would not have been possible without substantial support and collaboration. I would like to express my deepest gratitude to Prathima Ma'am for her continuous guidance, strategic oversight, and for fostering an environment of technical excellence within the CBIT community. Furthermore, I extend my sincere thanks to my teammates and collaborators, Vishnu Vardhan and Inayat; our extensive discussions regarding structural optimization, gradient flow, and the nuances of model deployment were invaluable to the successful execution of these experiments.

REFERENCES

[1] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Tech Report, 2009.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016.

[3] F. Chollet et al., "Keras: Deep Learning for humans," GitHub repository, 2015.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015.

[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," JMLR, 2014.